

CS2109S: Introduction to AI and Machine Learning

Lecture 7: Regularization, Kernels, and Dual Models

10 March 2026

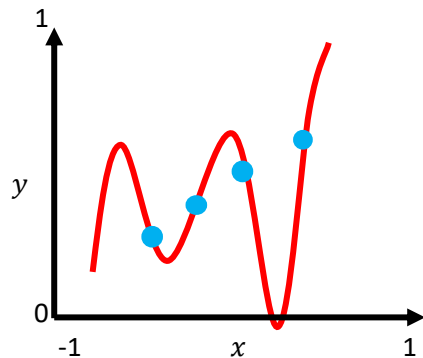
Outline

- **Regularization**
 - Overfitting
 - Regularization
 - Learning with regularization
- **Dual models and Kernel Method**
 - Dual formulation of linear regression
 - Transformed features
 - Kernel functions
- **Hyperplane classification model**
 - Hyperplanes, normal vectors.
 - How far is a point from the hyperplane?
 - Dual model.
 - Beyond the scope: Support Vector Machine (SVM).

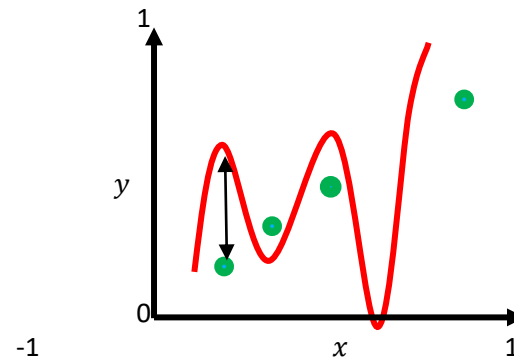
Overfitting: The Problem

Overfitting: Sometimes the model is too complex for the underlying ground truth. It learns the training data *too* well, including its noise and random fluctuations.

The model performs exceptionally well on training data but often fails on new data as it does not represent the underlying ground truth.



Training
The **model** “fits” all four **training data points** perfectly



Evaluation
The **trained model** fails on all four **test data points**

Overfitting: A Potential Cause

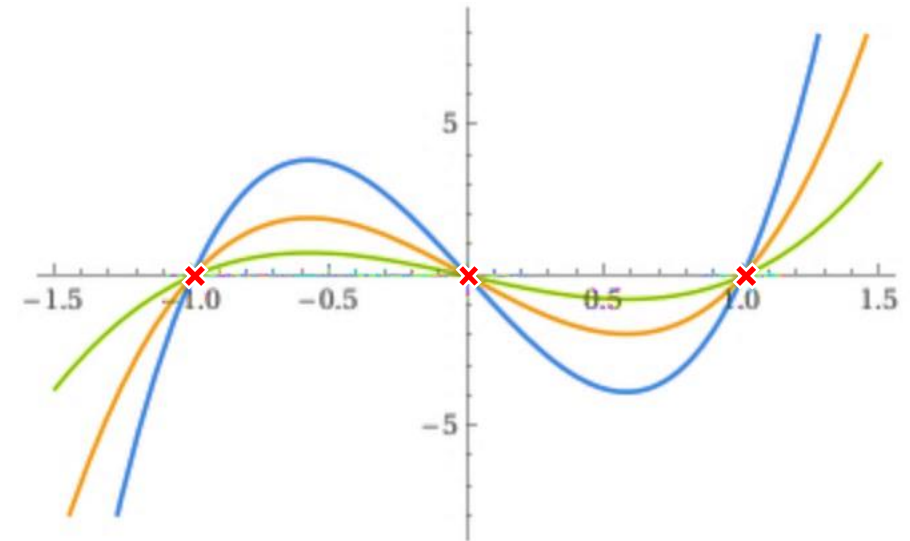
Consider a model $h_w(x) = w_1x + w_2x^3$ and a dataset

$$\{(x^{(i)}, y^{(i)})\} = \{(-1,0), (0,0), (1,0)\}$$

When $w_1 = -w_2$, increasing $|w_1|, |w_2|$ will increase the oscillation (non-linearity) while “fitting” the same three points.

Observation: overfitting is (often) associated with large-magnitude weights.

How do we measure the size of a weight vector?
What to do?



— $-10x + 10x^3$

— $-5x + 5x^3$

— $-2x + 2x^3$

Background: L_p -norm

The L_p -norm (or p-norm) is a function that measures the “size” of a vector.

For $p \geq 1$ and a vector w of dimension $d + 1$, $w = [w_0, w_1, \dots, w_d]^T$, the L_p norm is defined as:

$$\|w\|_p = \left(\sum_{j=0}^d |w_j|^p \right)^{\frac{1}{p}}$$

Example: L_2 -norm = $\sqrt{\sum_{j=0}^d w_j^2}$. Also known as Euclidean norm.

Regularization

Regularization is a technique of introducing constraints to a loss function to prevent a model from becoming too complex.

Regularization is achieved by augmenting a loss function $J(\mathbf{w})$, such as MSE, with a **penalty function** or **regularizer** $R(\mathbf{w})$, which is often (not always) defined using an L_p -norm.

$$J_{reg}(\mathbf{w}) = J(\mathbf{w}) + \lambda R(\mathbf{w})$$

where $\lambda \geq 0$ is the regularization parameter which controls the strength of the penalty. This is often called the **regularized loss function**.

Regularizer

Common regularizers:

- L_1 -norm (Manhattan norm)

$$R_{L_1}(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{j=0}^d |w_j|$$

- L_2 -norm (Euclidean norm)

$$R_{L_2}(\mathbf{w}) = \|\mathbf{w}\|_2 = \sqrt{\sum_{j=0}^d w_j^2}$$

In practice, we simplify it to $\frac{1}{2} \sum_{j=0}^d w_j^2$

- L_∞ -norm (Max norm)

$$R_{L_\infty}(\mathbf{w}) = \|\mathbf{w}\|_\infty = \max_j |w_j|$$

Common use cases:

- **Lasso regression:**
linear regression + L_1 -norm
- **Ridge regression:**
linear regression + L_2 -norm
- **Elastic Net:**
linear regression + L_1 -norm + L_2 -norm

Regularization – Example

Consider a model $h_{\mathbf{w}}(x) = w_1x + w_2x^3$ and a dataset

$$\{(x^{(i)}, y^{(i)})\} = \{(-1,0), (0,0), (1,0)\}$$

Suppose that we use the L1 regularizer:

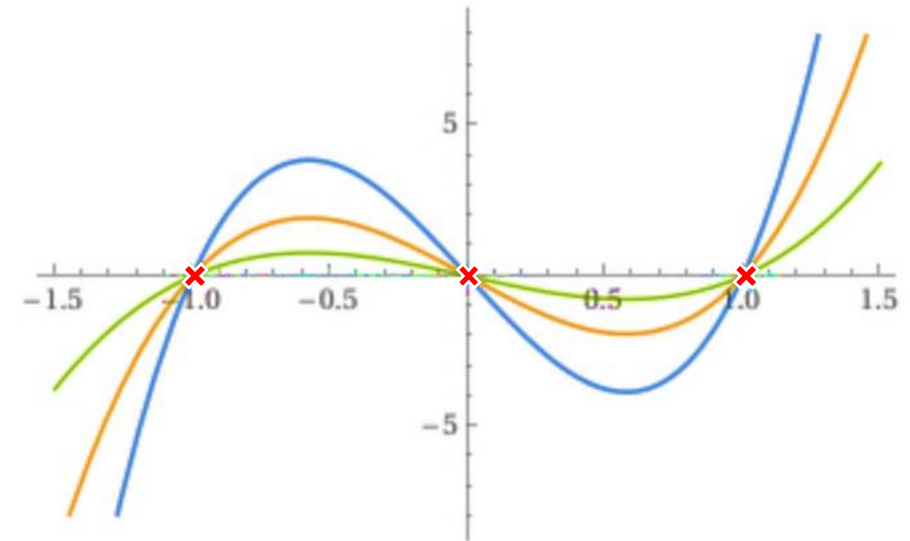
$$J_{L_1}^{MSE}(\mathbf{w}) = J_{MSE}(\mathbf{w}) + \lambda \sum_{j=0}^d |w_j|$$

The model can “fit” the data perfectly.

- We readily obtain $J_{MSE}(\mathbf{w}) = 0$.

Small weights make the regularized loss smaller.

- Smaller-weight model is “simpler” and has less oscillation.
- Simpler model that “fits” the data well likely captures the ground truth better.
- Here, $J_{L_1}^{MSE}(\mathbf{w})$ is zero when weights are zero.



— $-10x + 10x^3$
— $-5x + 5x^3$
— $-2x + 2x^3$

$$\|\mathbf{w}\|_1 = |10| + |10| = 20$$

$$\|\mathbf{w}\|_1 = |5| + |5| = 10$$

$$\|\mathbf{w}\|_1 = |2| + |2| = 4$$

Learning with Regularization

Objective: minimize the regularized loss.

$$J_{reg}(\mathbf{w}) = J(\mathbf{w}) + \lambda R(\mathbf{w})$$

Derivative:

$$\frac{\partial}{\partial w_j} J_{reg}(\mathbf{w}) = \frac{\partial}{\partial w_j} J(\mathbf{w}) + \lambda \frac{\partial}{\partial w_j} R(\mathbf{w})$$

The minimum of this function is when $\frac{\partial}{\partial w_j} J_{reg}(\mathbf{w}) = 0$ for all w_j .

For **lasso regression** (MSE + L_1 -norm):

$$\frac{\partial J_{L_1}^{MSE}(\mathbf{w})}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)} + \lambda \frac{\partial |w_j|}{\partial w_j}$$

Requires the **subgradient** $\frac{\partial |w_j|}{\partial w_j}$ which is beyond the scope of this class.

For **ridge regression** (MSE + L_2 -norm):

$$\frac{\partial J_{L_2}^{MSE}(\mathbf{w})}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j$$

Background: Identity Matrix

The identity matrix is a diagonal matrix with diagonal values of 1.

$$\mathbb{I} = \begin{bmatrix} 1 & 0 & \ddots & 0 \\ 0 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & 1 \end{bmatrix}$$

Given a scalar λ and identity matrix $\mathbb{I}$,

$$\lambda \mathbb{I} = \begin{bmatrix} \lambda & 0 & \ddots & 0 \\ 0 & \lambda & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & \lambda \end{bmatrix}$$

Suppose that you have a vector $w \in \mathbb{R}^{d+1}$,

$$w = \mathbb{I}w$$

$$\begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} = \begin{bmatrix} 1 & 0 & \ddots & 0 \\ 0 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & 1 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

$$\begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix} = \begin{bmatrix} 1w_0 + 0w_1 + \cdots + 0w_d \\ 0w_0 + 1w_1 + \cdots + 0w_d \\ \vdots \\ 0w_0 + 0w_1 + \cdots + 1w_d \end{bmatrix}$$

Normal Equation for Ridge Regression

The best linear regression model is the one that has the **minimum** loss, which is when the **partial derivative**

$$\frac{\partial J_{L_2}^{MSE}(\mathbf{w})}{\partial w_j} = 0$$

for all w_i .

Find the best $\mathbf{w}$ by solving a closed-form expression:

$$\mathbf{w} = (X^T X + \lambda \mathbf{I})^{-1} X^T Y$$

$$\frac{\partial J_{L_2}^{MSE}(\mathbf{w})}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)} + \lambda w_j = 0$$

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \dots & x_d^{(n)} \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

Express this with
vectors and matrices

$$X^T (X\mathbf{w} - Y) + n\lambda \mathbf{I} \mathbf{w} = 0$$

n can be absorbed by λ :
Let $\lambda \leftarrow n\lambda \geq 0$

$$X^T (X\mathbf{w} - Y) + \lambda \mathbf{I} \mathbf{w} = 0$$

Solve for $\mathbf{w}$

$$\mathbf{w} = (X^T X + \lambda \mathbf{I})^{-1} X^T Y$$

L_2 Regularization – Analysis

Theorem (Invertibility): Let X be an $n \times d$ matrix and $\mathbb{I}$ be the $d \times d$ identity matrix. For any real number $\lambda > 0$, the matrix $X^T X + \lambda \mathbb{I}$ is invertible.

Theorem (Solution Uniqueness): For any $\lambda > 0$, ridge regression (linear regression with L_2 regularization) has a unique, closed-form solution given by $\mathbf{w} = (X^T X + \lambda \mathbb{I})^{-1} X^T Y$.

Theorem (L_2 induces weights shrinkage): For any $\lambda > 0$, the L_2 -norm of the L_2 regularized weight vector is strictly smaller than the non-regularized weight vector, assuming the solution is non-zero.

L_1 Regularization – Analysis

Fact: A feature is included in the model if it has non-zero weight.

Theorem (L_1 penalty):

If a feature has non-zero weight, the L_1 regularizer will penalize the feature with a constant penalty of λ (in terms of gradient).

Theorem (L_1 induces sparsity):

For a feature to be included in the model, its contribution to reducing the MSE loss must be **exactly equal** to the regularization penalty λ

- Intuitive explanation: For the optimal model, the gradient of the loss must be zero. Hence the λ from the subgradient must be compensated by the gradient of the MSE.

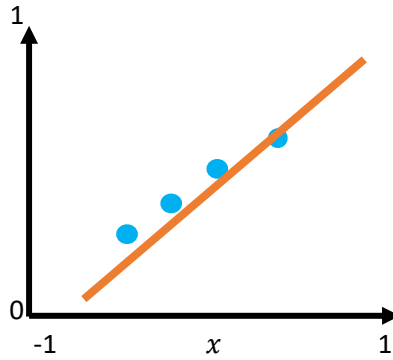
If a feature's contribution is **less than** the penalty λ , the optimization will prefer to set its weight to **exactly zero** rather than pay the L_1 penalty, inducing sparsity.

Outline

- Regularization
 - Overfitting
 - Regularization
 - Learning with regularization
- **Dual models and Kernel Method**
 - Dual formulation of linear regression
 - Transformed features
 - Kernel functions
- Hyperplane classification model
 - Hyperplanes, normal vectors.
 - How far is a point from the hyperplane?
 - Dual model.
 - Beyond the scope: Support Vector Machine (SVM).

Q: What determines the “optimal” linear model?

- Hint:



- Hint: $w = (X^T X + \lambda I)^{-1} X^T Y$

➤ The training data points!

➤ Hence, $w = \text{function}(??)$

➤ $w = \text{function}(x^{(1)}, \dots, x^{(n)})$

Recall: Feature Transformation

We can take a d -dimensional feature vector and transform it into a M -dimensional feature vector. Usually $M \geq d$.

$$x \in \mathbb{R}^d \quad \rightarrow \quad \phi(x) \in \mathbb{R}^M$$

Here,

- The function ϕ is called a **feature transformation** or **feature map**,
- The vector $\phi(x)$ is called **transformed features**.

Recall: What was the purpose of feature transformations?

The Problem with Feature Transformation

Example: Polynomial feature transformation: ϕ_{pk} .

For 2 features and degree 2: **6 transformed features**

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \phi_{p2}(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1x_2]^T$$

In general, for d features and degree k transformation: $\binom{k+d}{k}$

For **3** features and degree 2: **10 transformed features**

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \rightarrow \phi_{p2}(x) = [1, x_1, x_2, x_3, x_1^2, x_2^2, x_3^2, x_1x_2, x_1x_3, x_2x_3]^T$$

For **100** features and degree **6**: **1.6 Billion transformed features**

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_{100} \end{bmatrix} \rightarrow \phi_{p6}(x) = [1, x_1, \dots, x_{100}, x_1^2, \dots, x_{100}^2, x_1x_2, \dots, x_1x_{100}, \dots, x_{99}x_{100}, \dots]^T$$

Implication: linear model will need to have 1.6 billion weights! i.e., $w_0, w_1, w_2, \dots, w_{1.6B}$

Can we have linear model whose **number of weights** are not determined by **the number of features**?

Background: Linear Combination of Vectors

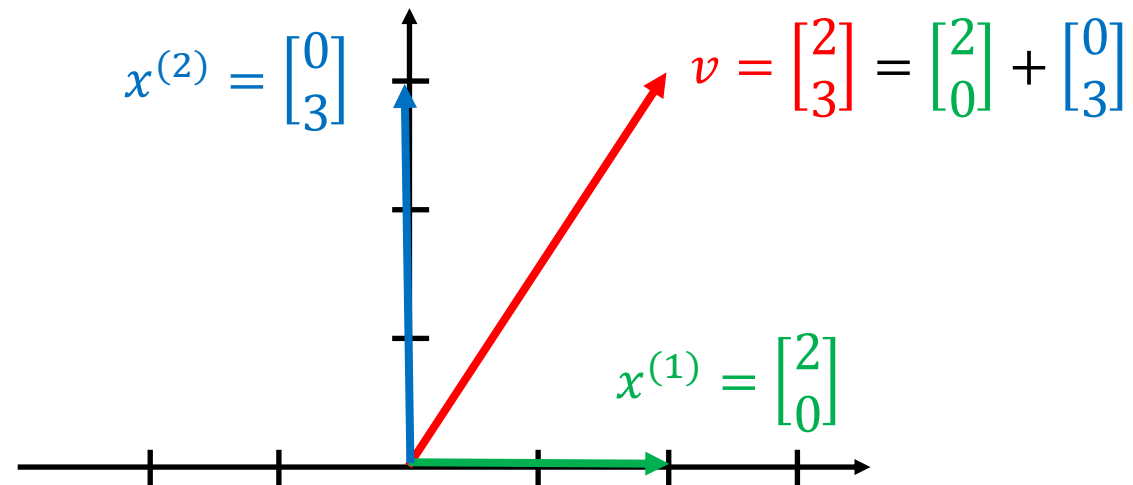
Let $x^{(1)}, x^{(2)}, \dots, x^{(n)}$ be n vectors. Let $\alpha_1, \alpha_2, \dots, \alpha_n$ be n real numbers.

A linear combination of these vectors, let's call it v , is computed as follows:

$$v = \alpha_1 x^{(1)} + \alpha_2 x^{(2)} + \dots + \alpha_n x^{(n)}$$

Example:

$(\alpha_1 = \alpha_2 = 1)$



Weights as A Linear Combination of Training Vectors

Theorem: For a linear model parameterized with a weight vector $\mathbf{w}$, the best weight vector $\mathbf{w}$ that minimizes the loss $J_{L_2}^{MSE}(\mathbf{w})$ can be expressed as:

$$\mathbf{w} = \sum_{j=1}^n \alpha_j \mathbf{x}^{(j)}$$

where α is a vector of real numbers $\alpha_1, \alpha_2, \dots, \alpha_n$ of dimension n .

Theorem: Proof Sketch

Consider normal equation.

Push-through matrix identity:

$$(X^T X + \lambda \mathbb{I})^{-1} X^T = X^T (X X^T + \lambda \mathbb{I})^{-1}$$

where X is a matrix, $\mathbb{I}$ identity matrix, and λ is a non-zero scalar.

This is a special form/application of the **Woodbury matrix identity**.

$$X = \begin{bmatrix} 1 & x_1^{(1)} & & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ 1 & \vdots & & \vdots \\ 1 & x_1^{(n)} & & x_d^{(n)} \end{bmatrix} \quad Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(n)} \end{bmatrix}$$

Using the push-through matrix identity on the expression given by the normal equation:

$$\begin{aligned} \mathbf{w} &= (X^T X + \lambda \mathbb{I})^{-1} X^T Y \\ &= X^T (X X^T + \lambda \mathbb{I})^{-1} Y \end{aligned}$$

Here, $(X X^T + \lambda \mathbb{I})^{-1} Y$ is an n -dimensional vector. Let's call this vector α .

Thus,

$$\mathbf{w} = X^T \alpha = \sum_{j=1}^n \alpha_j x^{(j)}$$

A more general result:

“Representer theorem”

Representer Theorem (Restricted, very informal): For a model parameterized with a weight vector w , in special cases the best weight vector w that minimizes a (regularized) loss over a training set of vectors $x^{(j)}$ can be expressed as:

$$w = \sum_{j=1}^n \alpha_j x^{(j)}$$

where α is a vector of real numbers $\alpha_1, \alpha_2, \dots, \alpha_n$ of dimension n .

Example: logistic regression with L_2 regularization and Support Vector Machines.

Dual Formulation of Linear Models

As consequence, linear model can be expressed using the α vector as follows

$$h_{\mathbf{w}}(x) = \mathbf{w}^T x = \sum_{j=1}^n \alpha_j x^{(j)T} x = h_{\alpha}(x)$$

The model:

- $h_{\mathbf{w}}(x)$ is called the **primal formulation of the model**.
- $h_{\alpha}(x)$ is called the **dual formulation of the model**.
 - Optimal model: primal = dual.

The same applies to logistic model (due to the representer theorem).

What is the loss function for the dual formulation?

How do we optimize the loss function with respect to α ?

Primal and Dual Objective

For ridge regression, we had the following objective optimizing for $\mathbf{w}$; this is called the **primal objective**.

$$J_{L_2}^{MSE}(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 + \lambda \sum_{i=0}^d w_i^2$$

For the dual formulation, we optimize the **dual objective** instead, i.e., minimize the loss over α :

$$J_{L_2}^{MSE}(\alpha) = \frac{1}{2n} \sum_{i=1}^n \left(\sum_{j=1}^n \alpha_j \mathbf{x}^{(j)T} \mathbf{x}^{(i)} - y^{(i)} \right)^2 + \lambda \sum_{i=0}^d \left(\sum_{j=1}^n \alpha_j x_i^{(j)} \right)^2$$

Theorem: Minimizing $J_{L_2}^{MSE}(\alpha)$ is equivalent to minimizing $J_{L_2}^{MSE}(\mathbf{w})$.

The same can be applied to logistic model.

The Problem with Feature Transformation

For 100 features and degree 6: 1.6 Billion transformed features

$$x = [1, x_1, \dots, x_{100}, x_1^2, \dots, x_{100}^2, x_1 x_2, \dots, x_1 x_{100}, \dots, x_{99} x_{100}, \dots]^T$$

Suppose that we use the transformed features x and the number of training data $n = 1000$.

Primal Formulation:

$$h_w(x) = \sum_{j=0}^d w_j x_j$$

Number of weights: 1.6 billion weights

Dual Formulation:

Quiz

Number of weights: ???

Addressing the Problem with Feature Transformation

For 100 features and degree 6: 1.6 Billion transformed features

$$x = [1, x_1, \dots, x_{100}, x_1^2, \dots, x_{100}^2, x_1 x_2, \dots, x_1 x_{100}, \dots, x_{99} x_{100}, \dots]^T$$

Suppose that we use the transformed features x and the number of training data $n = 1000$.

Primal Formulation:

$$h_w(x) = \sum_{j=0}^d w_j x_j$$

Number of weights: 1.6 billion weights

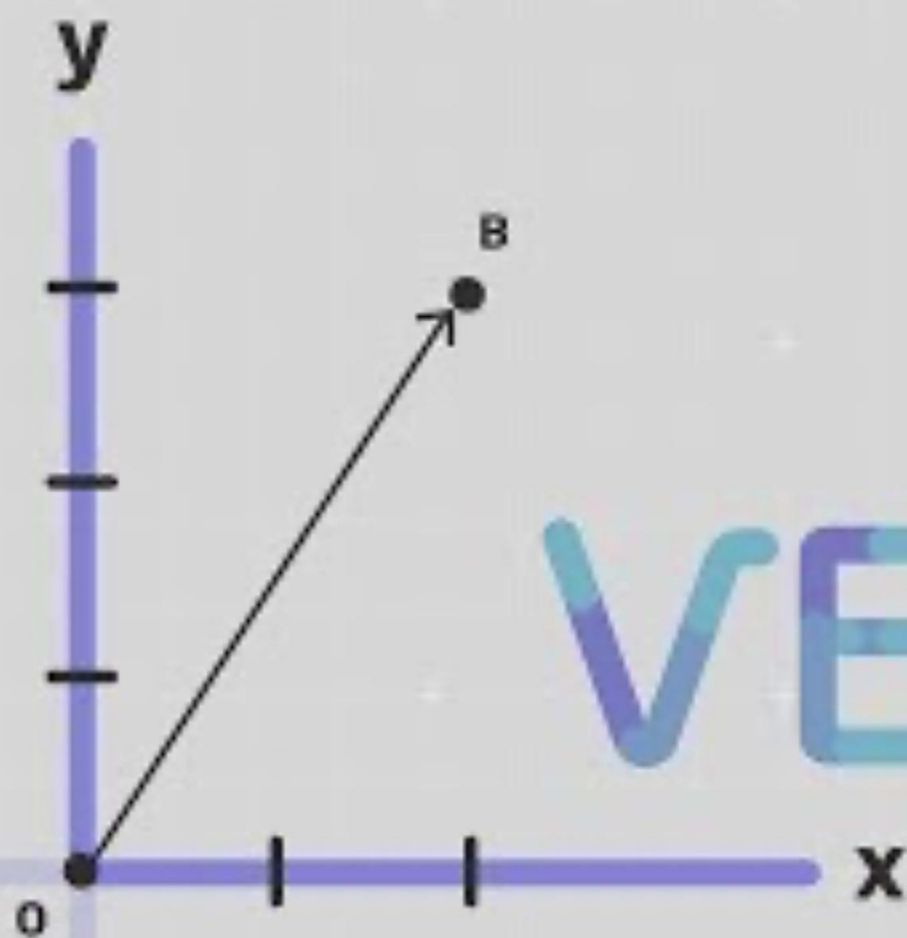
Dual Formulation:

$$h_\alpha(x) = \sum_{j=1}^n \alpha_j x^{(j)T} x$$

Number of weights: 1,000 weights

Wait, there's more benefit!

Break



WHAT ARE VECTORS?

Background: Inner Product and Dot Product

Many vector spaces are equipped with an *inner product*, denoted by $\langle u, v \rangle$ for u, v in the vector space.

Special case: In Euclidean vector space, the inner product is the dot product.

$$\langle u, v \rangle = u \cdot v = u^T v$$

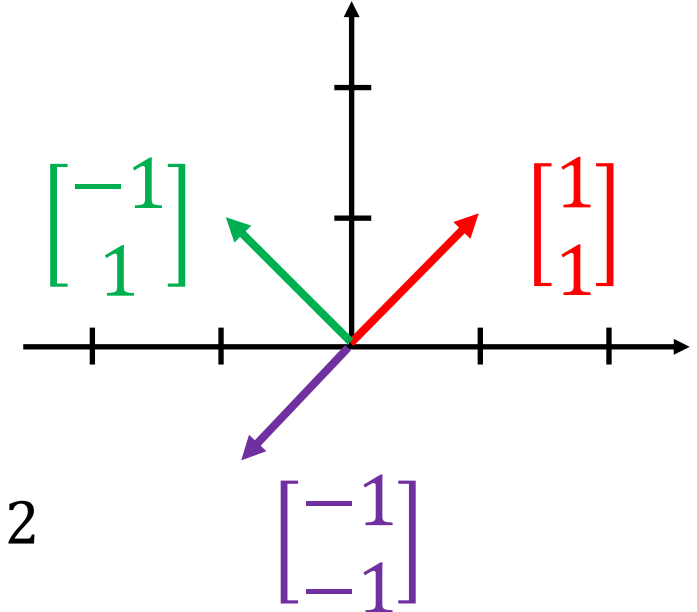
Example:

$$u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

$$\langle u, v \rangle = u \cdot v = u_1 v_1 + u_2 v_2$$

Dot product gives the similarity of two vectors:

$$\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 2 \quad \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 0 \quad \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} = -2$$

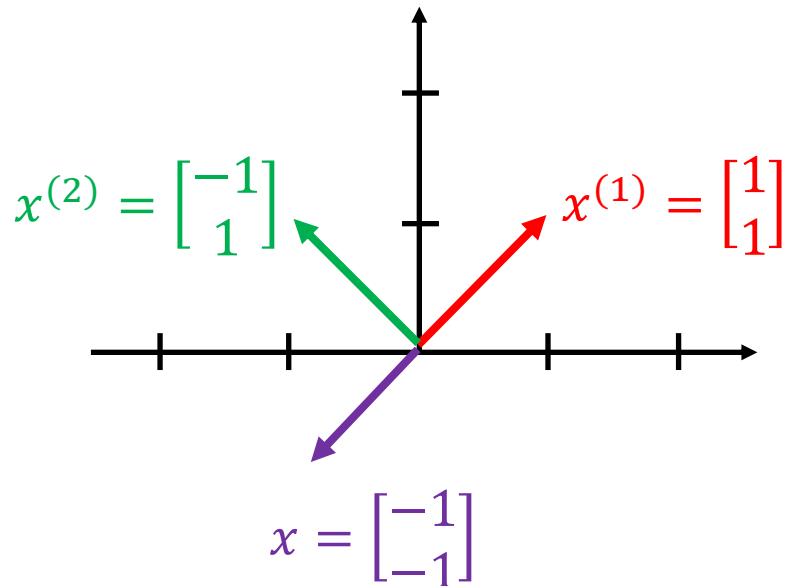


Note that general vector spaces can also be infinite-dimensional.

Dual Model with Inner Product

The dual formulation can be written in a more general form using inner product as follows

$$h_{\alpha}(x) = \sum_{j=1}^n \alpha_j x^{(j)T} x = \sum_{j=1}^n \alpha_j \langle x^{(j)}, x \rangle$$



To evaluate a given hypothesis at an arbitrary point x , compute the similarities (inner products) and sum the terms using α_j .

Example:

$$\begin{aligned} \alpha_1 &= 5 \\ \alpha_2 &= 11 \end{aligned}$$

$$h_{\alpha}(x) = 5 * (-2) + 11 * (0) = -10$$

Dual Model with Feature Transformation

The dual formulation can be written in a more general form using inner product as follows

$$h_{\alpha}(x) = \sum_{j=1}^n \alpha_j x^{(j)T} x = \sum_{j=1}^n \alpha_j \langle x^{(j)}, x \rangle$$

Definition (Kernel function): For any feature mapping function ϕ , the function $k_{\phi}(u, v) = \langle \phi(u), \phi(v) \rangle$, for any vectors u and v , is called a kernel function, or kernel.

Interpretation: A kernel is a measure of similarity of transformed features.

Suppose that you use the transformed feature $\phi(x)$ as input, your model becomes

$$h_{\alpha}(\phi(x)) = \sum_{j=1}^n \alpha_j \langle \phi(x^{(j)}), \phi(x) \rangle = \sum_{j=1}^n \alpha_j k_{\phi}(x^{(j)}, x) = h_{\alpha}^{\phi}(x)$$

Kernel Trick and Kernel Machine

The dual formulation with kernel is given as:

$$h_{\alpha}^{\phi}(x) = \sum_{j=1}^n \alpha_j k_{\phi}(x^{(j)}, x)$$

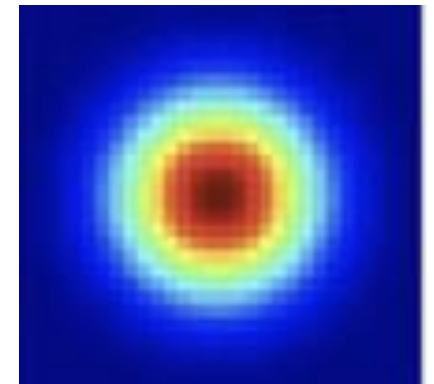
Notice that you do not have to expand $\phi(x)$ explicitly; simply compute kernel between $x^{(j)}$ and x . This is called **kernel trick**. A model that uses this trick is called **kernel machine**.

Is computing kernel cheaper than expanding and using the explicit transformed features?

Common Kernels

Computing kernel is very efficient!

- Polynomial degree k kernel
 - **Feature map:** Polynomial combinations of the features with degree $= k$.
 - Example (2 features, degree = 2): $\phi_{p2}([x_1 \ x_2]^T) \rightarrow [x_1 x_2, x_2 x_1, x_1^2, x_2^2]^T$
 - **Kernel:** $k_{pk}(u, v) = \phi_{pk}(u)^T \phi_{pk}(v) = (u^T v)^k$
- Gaussian kernel with variance s^2
 - Also known as **radial basis function (RBF)** kernel
 - **Kernel:** $k_{RBF}(u, v) := e^{-\frac{\|u-v\|^2}{2s^2}}$
 - **Feature map:** An infinite list of scaled powers of the input features
 - Example (1 feature): $\phi_{RBF}(x) = e^{-sx^2} \left[1 \quad \frac{\sqrt{2s}}{1!} x \quad \frac{\sqrt{(2s)^2}}{2!} x^2 \quad \frac{\sqrt{(2s)^3}}{3!} x^3 \quad \dots \right]^T$
- String kernel, chi-squared kernel, tanh kernel



Kernels: What comes first?

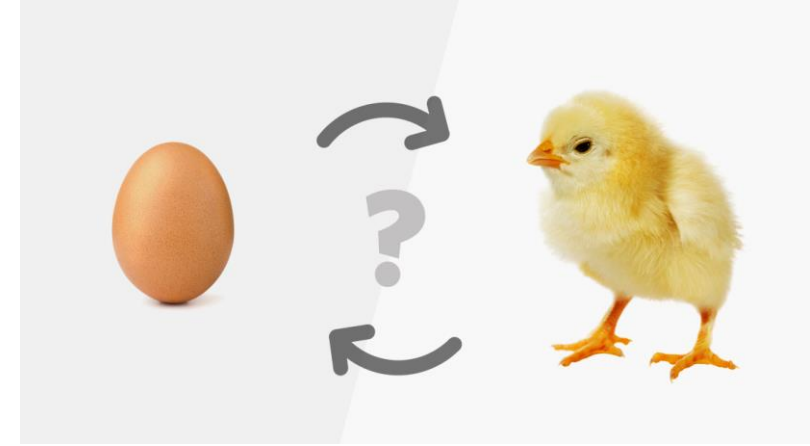


Image credit: LinkedIn

Feature transformation or Kernel function?

- For some kernels, e.g., polynomial kernel, we know the feature map first and the feature map induces the kernel.
- For some other kernels, e.g., RBF kernel, the kernel is created first. We cannot even practically compute the feature map because there will be infinite number of features.

Mercer's Theorem (Informal) For any valid kernel k_ϕ , there must exist a vector space with a feature mapping ϕ .

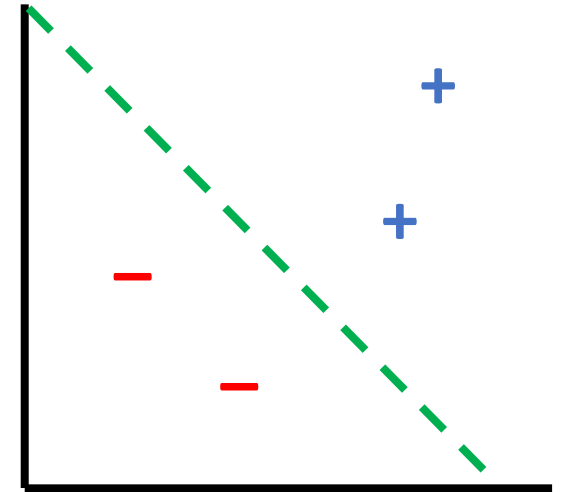
- Valid kernel = *continuous symmetric positive-definite* kernel (beyond the scope of this course).

Outline

- Regularization
 - Overfitting
 - Regularization
 - Learning with regularization
- Dual models and Kernel Method
 - Dual formulation of linear regression
 - Transformed features
 - Kernel functions
- **Hyperplane classification model**
 - Hyperplanes, normal vectors.
 - How far is a point from the hyperplane?
 - Dual model.
 - Beyond the scope: Support Vector Machine (SVM).

Hyperplane Classification Model

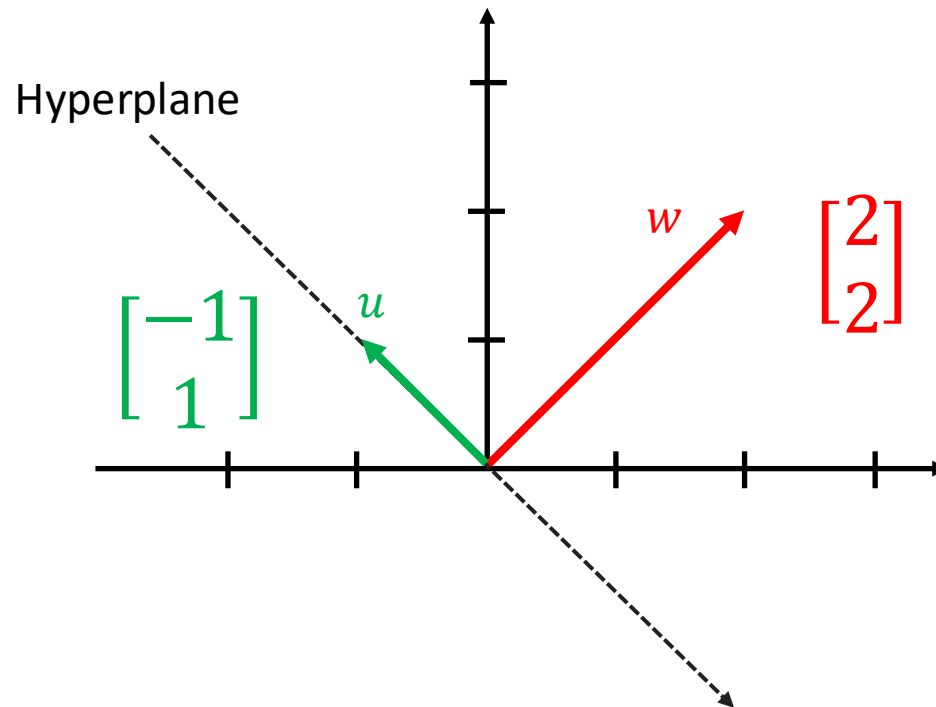
- So far, our classification model is based on predicting probability
 - E.g., logistic regression
- A different kind of classification model: predicts whether a point is on one or the other side of the decision boundary (hyperplane).
- One of the models is the most popular dual model and kernel machine.



Hyperplane: Definition

Disclaimer: in this lecture, for simplicity, we will only talk about hyperplane that passes through the origin at 0, called *zero-offset hyperplane*. Offset will be discussed in tutorial.

Definition: The zero-offset hyperplane is defined by a normal vector w and contains all vectors u for which $w^T u = 0$

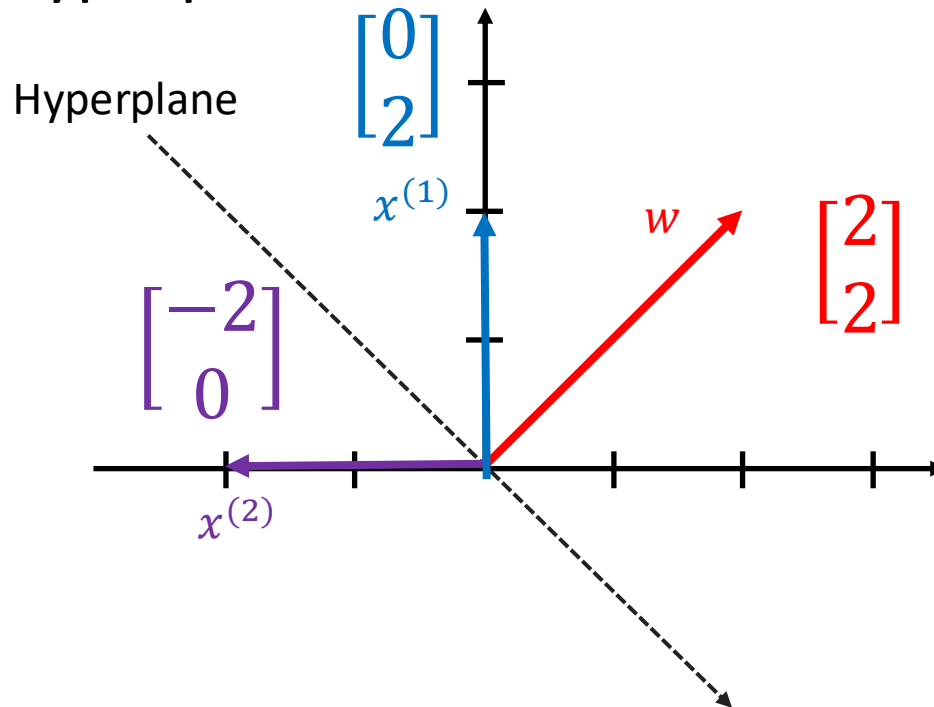


$$\begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = 0$$

Note: hyperplane = decision boundary.

Hyperplane: Prediction

A hyperplane can be used to make a prediction using $\text{sign}(w^T x)$. The sign of $w^T x$ tells us which side a point x is with respect to the (zero-offset) hyperplane.



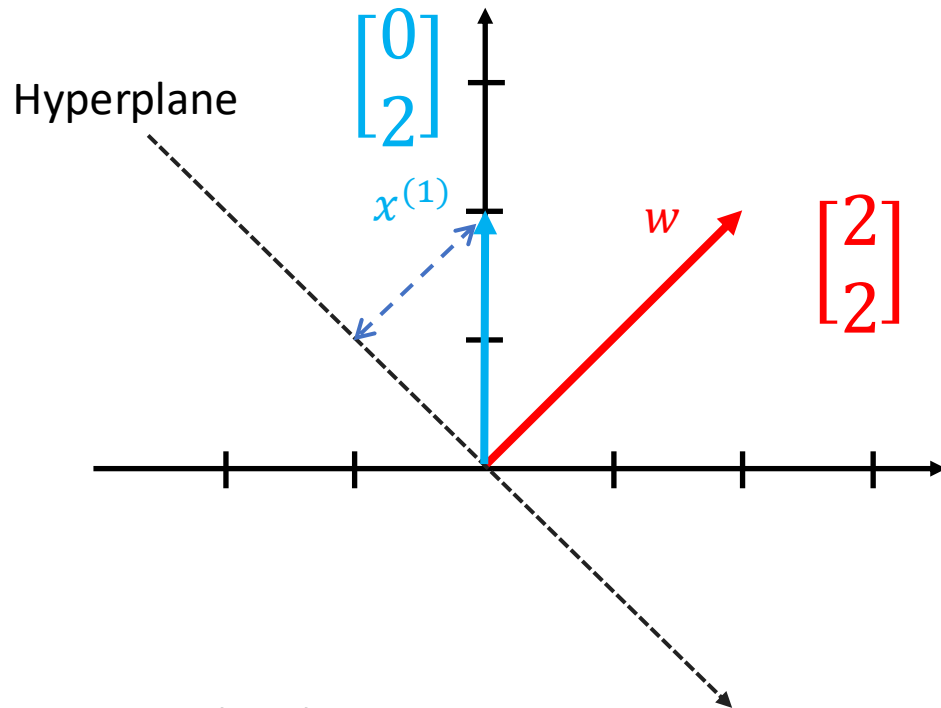
$$\begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = 4 \rightarrow +1$$

$$\begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} -2 \\ 0 \end{bmatrix} = -4 \rightarrow -1$$

Review: Distance to Hyperplane

Distance of a point x to a (zero-offset) hyperplane:

Compute $|w^T x|$ and divide the result by the length of w : $\frac{|w^T x|}{\|w\|}$.



$$\begin{bmatrix} 2 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = 4$$

$$\frac{4}{\sqrt{8}} = \sqrt{2} \approx 1.41$$

Hint: We can think of $\frac{|w^T x|}{\|w\|}$ as the length of the projection of x onto w .

Data

Suppose:

- We are given n data points.
- Each data point consists of **features** and a **target** variable.
- Each data point has d features and they are described by **real numbers**.
- The target is $\{-1, 1\}$, where -1 is “negative” class and 1 is “positive” class.
- Data set:

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\},$$

Hyperplane Classification Model

Given an input vector x of dimension d , a **hyperplane classification model** is a model with the following formulation:

$$h_{\mathbf{w}}(x) = \text{sign}(\mathbf{w}^T x)$$

where $\mathbf{w}$ is a vector of dimension d , and the sign function is defined as:

$$\text{sign}(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{if } z < 0 \end{cases}$$

Hyperplane Classification Dual Model

Via the representer theorem, the **dual formulation** of the hyperplane classification model is given by

$$h_{\alpha}(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i x^{(i)} \cdot x \right)$$

What happens when we include feature transformations?

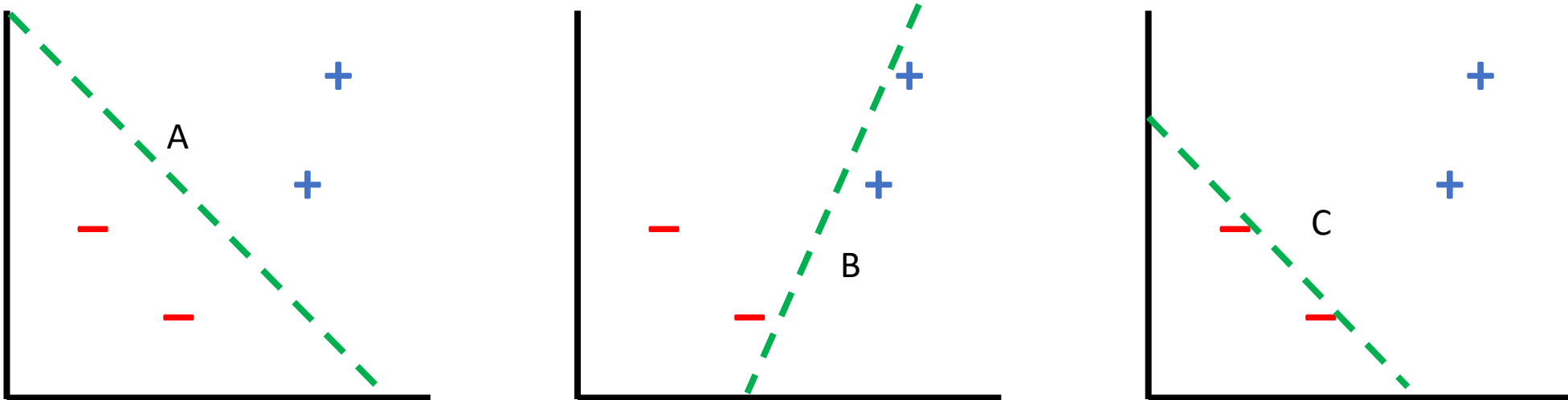
$$h_{\alpha}^{\phi}(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i k_{\phi}(x^{(j)}, x) \right)$$

Best Hyperplane Classification Model

So far, we choose the best classification model based on the ability to classify datapoints correctly.

However, there are **many hyperplanes** that achieve this.

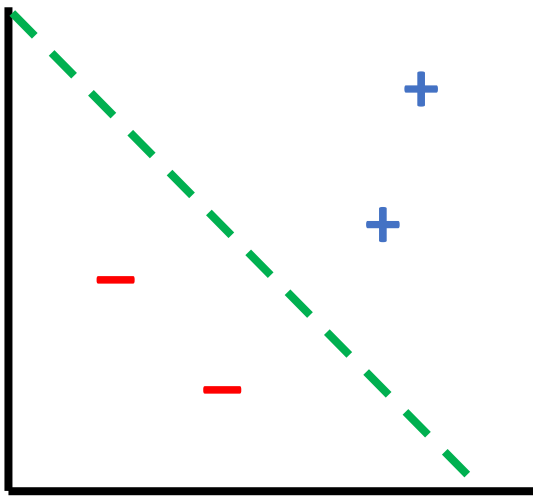
Which one is the best?



Support Vector Machines (SVM)

“Optimal” hyperplane classification model:

- Compute the best decision boundary.



$$h_{\alpha}^{\phi}(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i k_{\phi}(x^{(i)}, x) \right)$$

Objective (Idea):

Maximize the distance between the closest datapoints to the hyperplane.

Most popular hyperplane model and kernel machine.

SVM

```
from sklearn import svm
```

```
X = [[0, 0], [1, 1]]
```

```
y = [0, 1]
```

```
model = svm.SVC()
```

```
model.fit(X, y)
```

```
model.predict([[2., 2.]]) # predict unseen data
```

```
model.dual_coef_ # obtains the  $\alpha^{(i)}$ 
```

Summary

- Regularization
 - Overfitting usually = large weights
 - Regularization: $+R(\mathbf{w})$
 - Trade-off fit and simple model
- Dual models and Kernel Method
 - Dual formulation: $\sum_{i=1}^n \alpha_i \langle x^{(i)}, x \rangle$
 - Kernel: $k_\phi(u, v) = \langle \phi(u), \phi(v) \rangle$
 - Feature map $\leftrightarrow$ kernels
- Hyperplane classification model

Model
With x as features

Model
With $\phi(x)$ as features

Model with **Kernel Trick**
With $\phi(x)$ mapping to
finite-dimensional
features

Model with **Kernel Trick**
With $\phi(x)$ mapping to
infinite-dimensional
features



Coming Up Next Lecture

- Neural networks

To Do

- **Lecture Training 7**
 - +550 EXP
 - +100 Early bird bonus